

SIT329 Advanced Embedded Systems

Task 2.0P: Introduction to the CMSIS libraries for ARM MCU programming

Introduction

The Arduino IDE environment allows access to MCU functions to control peripherals such as GPIO pins using a set of standard functions e.g. *DigitalWrite()* and *AnalogRead()*. These functions are defined in Arduino libraries, making it easy to use but at the same time restrictive if you decide to build your own hardware platform based on ARM MCUs. For ARM Cortex M based MCUs, low level programming is done by accessing the Common Microcontroller Software Interface Standard (CMSIS) API libraries. Learning the libraries available for use in the CMSIS allows you to extend your ability to program ARM products outside the ARDUINO ecosystem, freeing you from the need to use the ARDUINO bootloaders.

Important Note

In this unit, we will focus on one of the basic ARM chipsets, namely the Cortex M0+. For simplicity, we select the Arduino Nano IoT 33 as an implementation board of choice for the discussion and tasks. The Cortex M0+ MCU on the Nano is the ATMEL SAMD21G18A and will be used as reference in workshops and online material. You may prefer to use other boards and IDEs to program ARM MCUs. This is allowable as long as the tasks are fulfilled and appropriately demonstrated.

Task Prerequisites:

1. Install Arduino IDE (version 2.0 or greater) on your PC which can be found at <https://www.arduino.cc/en/software>. Alternatively, search for Arduino IDE using your browser.
2. Install the library Arduino SAMD boards (32 bits ARM Cortex M0+) to have access to the Arduino Nano 33 IoT board.
3. Purchase or access to the Arduino Nano 33 IoT board from the Makerspace, and obtain the hardware list provided on CloudDeakin unit site. Please contact A/Prof. Daniel Lai (daniel.lai@deakin.edu.au) or Dr Clement Asogwa (Clement.Asogwa@deakin.edu.au) to borrow kits.
4. Download the datasheet for the SAMD21G18A MCU for your reference by searching Google, or use the following weblink.
https://ww1.microchip.com/downloads/en/DeviceDoc/SAM_D21_DA1_Family_DataSheet_DS40001882F.pdf

If you decide to use another development board for ARM Cortex M0+ or another Cortex architecture, then you will need to follow the appropriate setups for that board. Limited support would be available from the teaching team, but you are encouraged to experiment as long as it does not disadvantage you in completing your tasks for this unit.

Task:

In this task, you will write your own C/C++ functions to directly access and control the GPIO pins of the SAMD21G18A MCU using CMSIS API commands, thus by passing the Arduino API layer. You will need to refer to the SAMD21G18A MCU datasheet to understand what GPIO registers are used, their function and what the register values physically refer to in hardware. The task is as follows:

i) Develop and write your own GPIO class that implements the following methods:

a) Configure any specific GPIO pin to act as an input or output (Mode)

An example function prototype is given as below (you are free to design your solution):

bool Config_GPIO (char PortNum, char PinNum, Mode)

PortNum should allow for the GPIO groups A, B etc.

PinNum refers to the position of the pin in the GPIO group

Mode refers to input or output, suitably defined

Example Use:

To configure GPIO PA21 as an output, we would call Config_GPIO(A, 21, OUTPUT).

b) Configure any specific GPIO pin to be HIGH or LOW

The function prototype is given as below:

bool Write_GPIO (char PortNum, char PinNum, bool State)

PortNum should allow for the GPIO groups A, B etc.

PinNum refers to the position of the pin in the GPIO group

State refers to HIGH or LOW, suitably defined in code

Example Use:

To set GPIO PA21 to high or 5V, we would call `Write_GPIO (A , 21 , HIGH)`.

- c) Read any specific GPIO pin (the return value is either LOW or HIGH). Follow the framework and convention that you have selected to use for a) and b).
- ii) Develop a simple blink program on your Arduino using only the functions that you have written. The GPIO pin to be referred to in the functions, are the pins on the SAMD21G18A MCU chip. For example, PA21 refers to pin 21 in Port A of the MCU. For the Arduino Nano 33 IoT board, you will see that pin PA21 on the MCU is connected to the board designated pin output D10. Now write the program to blink the LED on Arduino Nano IoT 33 (pin13 on Arduino) every 1 second.

These methods look similar to `pinMode()`, `digitalWrite()` and `digitalRead()`, with the exception that the pin numbers refer directly to the pins of the MCU used, and not the pin labels on the Arduino Nano 33 IoT board. The implementation of the method including any Definitions is open to your preferences.

Task Submission Requirements:

A pdf file is required with answers to the Task requirements above, written in report format. Answer any questions in the discussion section of your report.

1. An introduction/definition of your GPIO class, including the names of your implementation files e.g. `GPIO_DEAKIN.h` and `GPIO_DEAKIN.c`.
2. Define your function prototypes clearly, show the input arguments, return value and the necessary definitions. Function prototypes are included in header files and the function implementations in the c files.
3. The full code listings for `GPIO_DEAKIN.h` and `GPIO_DEAKIN.c` should be included with appropriate comments. Do not take a photo of your screen. The full code listing for your blink program should also be included. This blink program may use the Arduino framework of `setup()` and `loop()`, but must use your GPIO methods to access the pins.
4. A link to a short video showing the following process – explain your code, compile your code and demonstrate that you are able to blink the LED using your own written code. You should be providing verbal explanation and not just showing mouse clicks.
5. Include any references used to complete this report.

Submit your work:

When you are ready, login to OnTrack and submit your pdf which consolidates all the items mentioned in the submission detail section above. Remember to save and backup your work.

Complete your work:

After your submission, your OnTrack reviewer (tutor) will review your submission and give you feedback in about 5 business days. Your reviewer may further ask you some questions on the weekly topics and/or about your submissions. You are required to address your OnTrack reviewer's questions as a form of task discussions. Please frequently login to OnTrack for the task Discuss/Demonstrate or Resubmit equivalent to fix your work (if needed) based on the feedback to get your task signed as Complete.